Oral Examination – Spoken Answers

Thesis ▪ Machine Learning and Perception ▪ AI for Simulation (rCFD)

Mochamad Bukhori Zainun 12438440 UK 066 993

Version of 10 October 2026, numbers from the thesis of the same date

How to use this book

The exam is a conversation of about one hour. First your talk and questions on it, then Machine Learning and Perception, then AI for Simulation. The examiner picks a topic and follows your answer with the next question.

Every answer has the same five parts.

- Core – one sentence that answers the question. Say it first. If you remember nothing else, say this.
- Explain – two or three short sentences that you can say out loud.
- Numbers – only the numbers you must know by heart.
- Derivation – the formula, built line by line. On the right of each line is the reason for the step. Write the lines on the board in this order.
- Summary – a short line of key words to memorise.

Five rules for the day

1. Say the core sentence first, then stop for a moment. Let the examiner ask for more.
2. Say the idea in words before you write the formula.
3. Write the simplest formula first. The examiner will move you to the one he wants.
4. End an answer with the next topic: "... and this is why we need regularization."
5. If you do not know, say what you know that is close: "I do not remember the constant, but the idea is ..."

Sources. Part A: the thesis of 10 October 2026 (`main.pdf`). Part B: the JKU script *Deep Learning and Neural Networks* (DLNN, WS24, and the joint script SS25), the LSTM lecture, and the list of past questions. Part C: the lecture *Numerical Methods in Fluid Mechanics* and its rCFD part (Prof. Pirker), and the figures of the thesis.

Contents

1 Thesis **2**

1.1 The talk in 15 minutes . . . 2

1.2 Questions on the thesis . . . 2

2 Machine Learning and Perception **9**

2.1 Empirical risk minimization and generalization . . . 9

2.2 From one neuron to a deep network . . . 10

2.3 Backpropagation and the delta error . . . 11

2.4 Gradient descent and optimizers . . . 13

2.5 Activation functions, regularization, initialization . . . 14

2.6 Convolutional networks . . . 15

2.7 Recurrent networks and LSTM . . . 15

2.8 Attention and Transformers 16
2.9 Classical machine learning in the same slot 17

3 AI for Simulation (Prof. Pirker) 20
3.1 Recurrence CFD as taught in the lecture 20
3.2 The figures of the thesis 21
3.3 CFD basics behind rCFD 24

1 Thesis

Thesis: *Data-based Numerical Simulation of Buoyant Jet Flow.* Supervisor: Prof. Stefan Pirker. Every number here is taken from the thesis of 10 October 2026. Equation and figure numbers refer to that version; the derivations follow Appendix D of the thesis.

1.1 The talk in 15 minutes

Build the talk on three claims, not on the chapters. One slide is about one minute.

Min	Slide	What you say
0–1	Title	Recurrence CFD records a flow once and replays it many times. I test it on a scalar that pushes back on the flow: temperature.
1–3	Problem	rCFD was built for passive scalars. Temperature drives the flow by buoyancy. What does the replay lose, and how much can I repair without solving the flow again?
3–4	Case	Closed tank, $75 \times 75 \times 15$ mm, 1 mm cells. Warm jet, 333 K at 0.02 m/s, for 315 s. $Re \approx 99$, laminar. Two lids: adiabatic and cooled.
4–6	Method	The preparation writes 600 frames. One replay step: recorded advection, face exchange, correction, conservation constraint. About 260 times cheaper.
6–8	Claim 1	No single mixing constant is best. The outlet sits just below the front, so a smeared front drains heat.
8–11	Claim 2	A bounded anti-diffusive correction, weighted to the vertical. Front 15.7 mm $\rightarrow$ 5.3 to 4.5 mm, reference 5.0 mm. Error of the mean temperature more than halved.
11–13	Claim 3	Under the cooled lid, on the fluid of the reference, the correction at $s \approx 0.55$ ends 0.06 K off on 9.62 K of cooling.
13–14	What remains	While the tank fills, the layers separate too slowly. A flux read from the recording reduces this error from 2.25 to 0.45 K, but the tank then ends 0.86 K too cold.
14–15	Take-home	A database belongs to a flow regime, not to a case. A numerical error at a front becomes an energy error near an outlet.

The thesis in one sentence. "I tested recurrence CFD on temperature, a scalar that drives its own flow. A bounded anti-diffusive correction removes the smearing of the front. The one defect that is left is missing transport, and a flux read from the recording can supply it."

1.2 Questions on the thesis

Q1.1 What problem did you solve?

Core. *I tested recurrence CFD on temperature, a scalar that drives its own flow.*
Explain. rCFD replays a recorded flow and moves a scalar with it. It was built for passive scalars, which do not change the flow. Temperature does change the flow. So I measured what the replay loses and how much I can repair without solving the flow again.
Numbers. Reference 569 min. One replay 2.16 min on average. About 260 times faster.

Summary: Passive scalar $\rightarrow$ active scalar. What is lost, what can be repaired.

Source: Thesis, Abstract and Chapter 1.

Q1.2 Why should anybody care?

Core. *Thermal processes run for hours, and full CFD of that time is too slow for daily use.*
Explain. rCFD is 260 times faster in my case. But a fast model is only useful if we know its error. My thesis measures this error for a case where temperature matters, and it shows which part can be repaired.

Summary: Fast is useful only with a known error.

Source: Thesis, Chapter 1.

Q1.3 Explain recurrence CFD in simple words.

Core. *A normal CFD run is recorded once, and the replay reuses the recorded motion again and again.*
Explain. The preparation solves the full equations and stores, frame by frame, which share of each cell moves to which cell. The replay applies these stored maps to the temperature. Velocity and pressure are never solved again, so it is cheap. One replay step has four stages.
Derivation.

$$\begin{aligned} \mathbf{T}^{*} &= \mathbf{M}^{(f_n)}\,\mathbf{T}^{n} && \text{(recorded map: move the heat)} \\ \mathbf{T}^{**} &= \mathcal{S}(\mathbf{T}^{*}) && \text{(face exchange: add mixing)} \\ \mathbf{T}^{***} &= \mathcal{C}(\mathbf{T}^{**}) && \text{(correction: my work)} \\ \mathbf{T}^{n+1} &= \mathcal{B}(\mathbf{T}^{***}) && \text{(conservation constraint: fix the total)} \end{aligned}$$

Together this is Eq. (2.10), $\mathbf{T}^{n+1} = \mathcal{B}(\mathcal{C}(\mathcal{S}(\mathbf{M}\mathbf{T}^{n})))$. Standard rCFD has $\mathcal{C} =$ identity.

Summary: Record once, replay many times. Map, swap, correct, balance.

Source: Thesis, Sections 2.2–2.7, Eq. (2.10).

Q1.4 Why is this AI, and not only CFD?

Core. *It is a data-based simulation: an expensive solver produces data, and a cheap model reuses it.*
Explain. The database plays the role of training data. Choosing the next frame is a nearest-neighbour search in a database of states, the method of analogues. My main finding is about generalization: a database belongs to a flow regime. A boundary condition that changes the regime makes the database out of distribution.

Summary: Database = training data. New regime = out of distribution.

Source: Thesis, Section 2.2 and Section 6.4.

Q1.5 What is the recurrence matrix, and is your flow recurrent?

Core. *The matrix compares every pair of recorded states, and my flow is not recurrent.*
Explain. A small distance means two states are similar, so the replay could jump between them. In my tank the matrix has two blocks, filling and settling, and no state comes back. So the replay plays the frames in order. It is still rCFD, but the saving comes only from not solving velocity and pressure.
Numbers. Median prominence 0.028. The method asks for 0.05 to 0.1.
Derivation.

$$\begin{aligned} R_{fg} &= \left\| \Phi^{(f)} - \Phi^{(g)} \right\| && \text{(distance between frames } f \text{ and } g\text{, Eq. (2.5))} \\ d(f,g) &= R_{fg} / \max R && \text{(normalise to } [0,1]\text{, Eq. (2.6))} \\ P_{\mathrm{rel}} &= \langle d_{\max} - d_{\min} \rangle / \bar{d} && \text{(depth of the dips over the plateau, Eq. (2.7))} \end{aligned}$$

Summary: Two blocks, no return, 0.028 → frames in order.
Source: Thesis, Section 2.4, Figure 2.2.

Q1.6 What is the face exchange, and what is f_{sd}?

Core. *After each advection the library moves a share of heat across every face, from warm to cold.*
Explain. A pure displacement does not mix, so this step adds mixing. f_{sd} is its strength per step. It has no unit, so it is not a diffusion coefficient, but it can be translated into one. The exchange can only reduce differences. It can never rebuild a smeared front.
Numbers. $f_{\mathrm{sd}} = 0.5$ gives 2.5×10^{-7} m^2/s, ten times the molecular value.
Derivation.

$$\Delta_{\mathrm{swap}} = f_{\mathrm{sd}}\, V\, \frac{T_{c_1} - T_{c_0}}{2} \qquad \text{(exchange of Eq. (2.11))}$$

$$Q_{\mathrm{swap}} = \rho c_p\, f_{\mathrm{sd}}\, c_s^3\, \frac{T_{c_1} - T_{c_0}}{2} \qquad \text{(heat of that amount, } V = c_s^3\text{)}$$

$$Q_{\mathrm{Fick}} = \rho c_p\, D\, \frac{T_{c_1} - T_{c_0}}{c_s}\, c_s^2\, \Delta t \qquad \text{(Fick's law over one face and one step)}$$

$$D_{\mathrm{eq}} = \frac{f_{\mathrm{sd}}\, c_s^2}{2\,\Delta t} \qquad \text{(set } Q_{\mathrm{Fick}} = Q_{\mathrm{swap}}\text{, Eq. (2.13))}$$

Summary: Swap = one-way mixing. Converts to $D_{\mathrm{eq}} = f_{\mathrm{sd}} c_s^2 / 2\Delta t$.
Source: Thesis, Section 2.6, Eqs. (2.11)–(2.13); Appendix D.1.

Q1.7 Why is there no single best mixing constant?

Core. *The two measures want different amounts of mixing, because the outlet sits just below the front.*
Explain. The mean temperature is best with little mixing. The height of the heat is best near $f_{\mathrm{sd}} = 0.5$. The more the replay smears the front, the more warm fluid reaches the outlet and leaves the tank.
Numbers. Mean-temperature error 1.07 K at little mixing. At $f_{\mathrm{sd}} = 3$ the tank ends 5.08 K too cold.
Summary: Outlet below the front → smearing drains heat.
Source: Thesis, Section 5.1.

Q1.8 Why not simply use negative diffusion to sharpen the front?

Core. *Negative diffusion is ill posed: small wiggles grow fastest.*
Explain. It does steepen the front, but it also amplifies every disturbance at the grid scale. In my 1D test a disturbance of 1 mK left the range of the data by more than 1 K after 21 steps. So I keep the useful part and stop the dangerous part with a limiter.
Derivation.

$$T_i^{n+1} = T_i^n + \nu\,(T_{i+1}^n - 2T_i^n + T_{i-1}^n) \qquad \text{(explicit diffusion step)}$$

$$T_i^n = G^n e^{\mathrm{i}ki\Delta x} \qquad \text{(one Fourier mode)}$$

$$G = 1 + \nu\,(2\cos k\Delta x - 2) = 1 - 4\nu \sin^2(k\Delta x/2) \qquad \text{(insert the mode)}$$

$$G = 1 + 4|\nu| \qquad \text{(shortest mode } k\Delta x = \pi,\ \nu < 0\text{)}$$

$|G| > 1$, so the shortest mode grows in every step.
Summary: Negative diffusion grows wiggles. Keep the steepening, limit the rest.

Source: Thesis, Section 4.2.2, Eq. (4.13), Figure 4.1.

Q1.9 Explain your limiter.

Core. *Each face may move heat up the gradient only as far as both cells stay inside their local bounds.*
Explain. For every cell I take the smallest and the largest value of the cell and its neighbours. Across a face, heat moves from the colder to the warmer cell. The amount is the smaller of the two headrooms, times a strength s. This is the limiter idea of flux-corrected transport, by Zalesak.
Derivation.

$$
\begin{aligned}
Q_h^+ = C_h\,(T_h^{\max} - T_h), \quad Q_c^- = C_c\,(T_c - T_c^{\min}) &\qquad \text{(headrooms, Eq. (4.15))}\\
\Delta q_f = s\,\min(Q_h^+, Q_c^-) &\qquad \text{(limited exchange, Eq. (4.16))}\\
C_i\,\Delta T_i \le n_f\,s\,Q_i^+ &\qquad \text{(at most } n_f \text{ faces bring heat)}\\
T_i + \Delta T_i \le T_i^{\max} \quad \text{if} \quad s \le 1/n_f = 1/6 &\qquad \text{(Eq. (4.17))}
\end{aligned}
$$

Summary: Smaller headroom times s. Conservative, no new extremum, safe for $s \le 1/6$.
Source: Thesis, Section 4.2.3; Appendix D.4.

Q1.10 Why the direction weight?

Core. *The front is horizontal, so the correction should act across it and not along it.*
Explain. Each face gets the weight $\cos^2$ of the angle between its link and the vertical. Faces across the front get one, faces along it get zero. Without the weight the correction roughens the front sideways. This weight is exactly the face coefficient of a diffusion that acts in one direction only.
Derivation.

$$
\begin{aligned}
K_f &= \hat{\mathbf{e}}^{\mathsf{T}}\mathbf{K}\,\hat{\mathbf{e}} && \text{(two-point face coefficient)}\\
&= K\,(\hat{\mathbf{e}}\cdot\hat{\mathbf{n}})^2 && \text{(rank-one tensor } \mathbf{K} = K\hat{\mathbf{n}}\hat{\mathbf{n}}^{\mathsf{T}})\\
&= K\,w_f^{\mathrm{dir}} && \text{(Eq. (4.21))}
\end{aligned}
$$

Summary: Act across the front: $\cos^2$ weight = one-direction diffusion.
Source: Thesis, Section 4.2.4, Eqs. (4.19), (4.21).

Q1.11 The recorded diffusivity did not help. Why?

Core. *The recorded values are larger than the explicit exchange can apply, so every face hits the cap.*
Explain. The library exchanges at most half a cell volume per face and step. The recorded diffusivity asks for about nine times more. So every face exchanges the maximum, and the field acts exactly like the constant $f_{\mathrm{sd}} = 0.5$. The replays are bit-identical, which proves it. A shorter step or an implicit exchange would let the field act.
Derivation.

$$
\begin{aligned}
4 \times \tfrac{1}{8} = \tfrac{1}{2} &\qquad \text{(at most four loops of 1/8 of a cell)}\\
\mathrm{Di} = D\,\Delta t/c_s^2 \le 0.5 &\qquad \text{(the cap as a diffusion number, Eq. (2.16))}
\end{aligned}
$$

Summary: Recorded D above the cap → acts like a constant.
Source: Thesis, Section 5.2.1, Figure 5.4.

Q1.12 What happened under the cooled lid?

Core. *On the fluid of the reference, the corrected replay ends 0.06 K from the reference.*
Explain. The reference fluid has the specific heat 1006.43 J/(kg K). A replay given the specific heat of water cools four times less for the same wall power, and ends 4.34 K too warm. On the right fluid the replay is close, and the correction puts the heat back in the right layer.
Numbers. Cooling depth 9.62 K. Without correction −0.66 K. With correction at $s \approx 0.55$: +0.06 K, end profile 0.41 K.
Derivation.

$$\Delta T_{\text{cell}} = -\frac{q\,\Delta t}{\rho\, c_p\, V} \qquad \text{(wall loss of one cell, Eq. (4.25))}$$

$$\frac{\Delta T_{\text{rep}}}{\Delta T_{\text{case}}} = \frac{c_{\text{case}}}{c_{\text{rep}}} = \frac{1006.43}{4182} = \frac{1}{4.155} \qquad \text{(same power, two specific heats)}$$

$$h_{\text{eff}} = G\, h\, \frac{c_{\text{case}}}{c_{\text{rep}}} \qquad \text{(Eq. (4.26))}$$

Summary: Same fluid on both sides first. Then the correction does the rest.
Source: Thesis, Sections 6.1–6.2; Appendix D.5.

Q1.13 What could not be repaired, and why?

Core. *While the tank fills, the layers separate too slowly, and no local rule can fix that.*
Explain. A local correction only moves heat between neighbours, so it can change the shape of the front but not carry heat from one layer to another. This is missing transport. A vertical flux read from the recording supplies that transport. It reduces the error, but the tank then ends too cold.
Numbers. Heterogeneity error over the warm inflow 2.25 K → 0.45 K. End 0.86 K too cold.
Derivation.

$$\rho c_p v T = \rho c_p v\, \bar{T}_{\text{b}} + \rho c_p v\, (T - \bar{T}_{\text{b}}) \qquad \text{(split } T \text{ at the band mean)}$$

$$F = \rho c_p v\, (T - \bar{T}_{\text{b}}) \qquad \text{(the recorded flux, Eq. (2.18))}$$

$$\langle F \rangle_{\text{b}} = \rho c_p \langle v' T' \rangle_{\text{b}} \qquad \text{(band mean: the eddy heat flux)}$$

Summary: Local repair fixes shape. Missing transport needs recorded transport.
Source: Thesis, Sections 6.5–6.6; Appendix D.2.

Q1.14 How do you know your results are correct?

Core. *Every replay is deterministic, and every run changes one setting against a control.*
Explain. A rerun gives the same trace, bit for bit. Replay and reference are compared at the same instants and over the same end window. The energy budget of each case is closed term by term.
Numbers. End window 580 to 599 s on both sides.
Summary: Deterministic, one change per run, same clock, closed budget.
Source: Thesis, Section 3.9 and Appendix B.5.

Q1.15 Why these error measures? Bias and RMSE?

Core. *The mean temperature says how much heat there is; the profile and the front width say where it is.*
Explain. A replay can be right on one and wrong on the other, so I report both. On a trace,

the bias gives the side and the RMSE gives the size. Their identity separates an offset from a wrong shape.
Derivation.

$$\begin{aligned}
\mathrm{RMSE}^2 &= \frac{1}{N}\sum_k e_k^2 && \text{(definition, Eq. (4.3))}\\
&= \frac{1}{N}\sum_k \big[(e_k - \mathrm{bias}) + \mathrm{bias}\big]^2 && \text{(add and subtract the bias)}\\
&= \sigma_e^2 + 2\,\mathrm{bias}\cdot 0 + \mathrm{bias}^2 && \text{($\sum_k (e_k - \mathrm{bias}) = 0$)}\\
&= \mathrm{bias}^2 + \sigma_e^2
\end{aligned}$$

Summary: How much heat versus where it is. $\mathrm{RMSE}^2 = \mathrm{bias}^2 + \mathrm{spread}^2$.
Source: Thesis, Section 4.1.2, Eq. (4.3).

Q1.16 Why is the flow laminar?

Core. *The jet enters at a Reynolds number of about 99, far below transition.*
Explain. So the equations are solved as they stand, with no turbulence model. The case file confirms that the turbulence model is off. Buoyancy is slightly stronger than inertia, so the jet bends up within a few inlet widths.
Numbers. $Re = UD/\nu \approx 99$ with $U = 0.02$ m/s, $D = 5$ mm. $Ri = 1.84$.
Summary: $Re \approx 99$: laminar. $Ri = 1.84$: the jet bends up.
Source: Thesis, Section 3.3, Table 3.2.

Q1.17 Why the Boussinesq approximation?

Core. *The density changes by only 1.5 per cent, so it matters only where gravity acts.*
Explain. Density is kept constant except in the buoyancy term. This buoyancy term is exactly the link that the replay cuts: the replay keeps the recorded motion, so the temperature can no longer change the velocity.
Derivation.

$$\begin{aligned}
\rho\,\mathbf{g} &\approx \rho_0\,\mathbf{g}(1 - \beta(T - T_0)) && \text{(linear in } T \text{ for small changes)}\\
&= \rho_0\,\mathbf{g} - \rho_0\,\beta\,(T - T_0)\,\mathbf{g} && \text{(second term = buoyancy)}
\end{aligned}$$

Summary: Constant density, except where gravity sees it. The replay cuts this term.
Source: Thesis, Section 2.1, Eqs. (2.1)–(2.3).

Q1.18 What is future work?

Core. *Let the recorded diffusivity act, split the conservation constraint, and switch databases by state.*
Explain. An implicit exchange or a shorter step would let the recorded diffusivity work. A balance per compartment would stop the constraint from putting heat back at the front. Several recordings, chosen by the replayed temperature, could cover more than one regime.
Summary: Implicit exchange, compartments, several databases.
Source: Thesis, Section 9.3.

Q1.19 Which papers influenced you most?

Core. *Lichtenegger and Pirker (2016) and Pirker et al. (2018) for rCFD, and Zalesak (1979) for the limiter.*

Explain. The first introduced recurrence CFD and the recurrence matrix. The second describes the face exchange and the conservation constraint. Zalesak gives the local bounds of flux-corrected transport that my limiter uses.

Summary: rCFD: Lichtenegger, Pirker. Limiter: Zalesak.

Source: Thesis, Chapter 2 and Section 4.2.

2 Machine Learning and Perception

Likely examiners: Prof. Klambauer or Prof. Hochreiter. The same core comes back almost every time: empirical risk minimization, gradient descent, backpropagation and the delta error. Then short questions on activations, regularization, optimizers and one architecture. Notation of the JKU script: input $\mathbf{x}_n$, label y_n, model $g(\mathbf{x};\mathbf{w})$, loss L, pre-activation s, activation $a = f(s)$, layer $[l]$.

2.1 Empirical risk minimization and generalization

Q2.1 Write down a supervised data set and the training framework.

Core. *We have N pairs of input and label, and learning means minimizing the average loss on them.*
Explain. The pairs are drawn independently from an unknown distribution. A model maps an input to a prediction, and a loss says how wrong one prediction is. The average loss on the training data is the empirical risk.
Derivation.

$$Z = \{(\mathbf{x}_n, y_n)\}_{n=1}^{N} \qquad \text{(training data, i.i.d. from } p(\mathbf{x}, y)\text{)}$$

$$R_{\text{emp}}(\mathbf{w}) = \frac{1}{N}\sum_{n=1}^{N} L(y_n, g(\mathbf{x}_n;\mathbf{w})) \qquad \text{(average loss)}$$

$$\hat{\mathbf{w}} = \arg\min_{\mathbf{w}} R_{\text{emp}}(\mathbf{w}) \qquad \text{(learning)}$$

Summary: Data, model, loss, average, minimize.
Source: DLNN script WS24, Ch. 3 and 5.1.

Q2.2 What is the generalization error? Why can we not minimize it directly?

Core. *It is the expected loss on new data, and we cannot compute it because the true distribution is unknown.*
Explain. So we minimize the empirical risk instead and estimate the generalization error on data we did not train on: a test set or cross-validation. The training loss is not a fair estimate, because the weights were chosen on that data.
Derivation.

$$R_{\text{gen}}(\mathbf{w}) = \mathbb{E}_{(\mathbf{x},y)\sim p}[L(y, g(\mathbf{x};\mathbf{w}))] \qquad \text{(expected loss)}$$

$$= \int L(y, g(\mathbf{x};\mathbf{w}))\, p(\mathbf{x}, y)\, \mathrm{d}\mathbf{x}\, \mathrm{d}y \qquad \text{(needs } p\text{, which is unknown)}$$

$$\approx \frac{1}{M}\sum_{m=1}^{M} L(y_m, g(\mathbf{x}_m;\mathbf{w})) \qquad \text{(test set, law of large numbers)}$$

Summary: Risk = loss on unseen data. Estimate it with a test set or cross-validation.
Source: DLNN script WS24, Ch. 5.2–5.3.

Q2.3 What is a loss function? Give examples.

Core. *A loss compares a label with a prediction and is zero when the prediction is right.*
Explain. It is never negative. We choose it to fit the task. Not every loss can be trained with gradient descent: the 0–1 loss has no useful gradient, so we use cross-entropy instead. Explain.

Examples: squared loss $\frac{1}{2}(y-\hat{y})^2$ for regression; binary cross-entropy $-y\log\hat{y}-(1-y)\log(1-\hat{y})$; categorical cross-entropy $-\sum_k y_k \log \hat{y}_k$; hinge loss $\max(0, 1 - y\, g(\mathbf{x}))$ for the SVM.

Summary: Non-negative, zero when right. 0–1 loss → use cross-entropy.

Source: DLNN script WS24, Sec. 4.1–4.4.

Q2.4 Show ERM on linear regression.

Core. *With a linear model and squared loss the risk is convex, so the minimum has a closed form.*

Explain. Set the gradient to zero and solve. Gradient descent reaches the same point step by step. For one sample the gradient is error times input, and the same pattern comes back in backprop.

Derivation.

$$
\begin{aligned}
R_{\text{emp}}(\mathbf{w}) &= \tfrac{1}{2}\|\mathbf{y} - X\mathbf{w}\|^2 && \text{(squared loss, all samples)}\\
\nabla_{\mathbf{w}} R_{\text{emp}} &= -X^{\mathsf{T}}(\mathbf{y} - X\mathbf{w}) && \text{(chain rule)}\\
0 &= X^{\mathsf{T}}\mathbf{y} - X^{\mathsf{T}}X\,\hat{\mathbf{w}} && \text{(gradient zero at the minimum)}\\
\hat{\mathbf{w}} &= (X^{\mathsf{T}}X)^{-1}X^{\mathsf{T}}\mathbf{y} && \text{(normal equations, pseudo-inverse)}
\end{aligned}
$$

Summary: Convex → closed form. Gradient = error × input.

Source: DLNN script WS24, Sec. 4.1.3–4.1.4.

Q2.5 Overfitting, underfitting, bias and variance.

Core. *Too simple a model has high bias, too complex a model has high variance.*

Explain. An underfitting model cannot follow the data. An overfitting model follows the noise, so the training error is low and the test error is high. Very large networks show double descent: after the point where they fit the training data exactly, the test error falls again.

Derivation.

$$
\begin{aligned}
y &= f(\mathbf{x}) + \varepsilon, \quad \mathbb{E}[\varepsilon] = 0,\ \mathrm{Var}[\varepsilon] = \sigma^2 && \text{(data model)}\\
\mathbb{E}[(y - \hat{g})^2] &= \mathbb{E}[(f + \varepsilon - \bar{g} + \bar{g} - \hat{g})^2] && \text{(add and subtract } \bar{g} = \mathbb{E}[\hat{g}])\\
&= \sigma^2 + (f - \bar{g})^2 + \mathbb{E}[(\hat{g} - \bar{g})^2] && \text{(cross terms have zero mean)}\\
&= \text{noise} + \text{bias}^2 + \text{variance}
\end{aligned}
$$

Summary: Simple → bias. Complex → variance. Large nets: double descent.

Source: DLNN script WS24, Sec. 5.4.

2.2 From one neuron to a deep network

Q2.6 Write the formula of a neuron.

Core. *A neuron takes a weighted sum of its inputs plus a bias, and passes it through a non-linear function.*

Explain. The weighted sum is the pre-activation, and the result is the activation. A layer does this for many neurons at once. The forward pass goes layer by layer from the input to the output.

Derivation.

$$
\begin{aligned}
s_i &= \sum_j w_{ij}\, a_j + b_i && \text{(pre-activation)} \\
a_i &= f(s_i) && \text{(activation)} \\
\mathbf{a}^{[l]} &= f\big(W^{[l]}\mathbf{a}^{[l-1]} + \mathbf{b}^{[l]}\big), \quad \mathbf{a}^{[0]} = \mathbf{x} && \text{(one layer; output } \hat{\mathbf{y}} = \mathbf{a}^{[L]}\text{)}
\end{aligned}
$$

Summary: Weighted sum, plus bias, through a non-linearity.
Source: DLNN script WS24, Eqs. (3.2)–(3.3), (4.59)–(4.62).

Q2.7 A deep network with and without activation functions?

Core. *Without non-linear activations the whole network collapses to one linear model.*
Explain. A product of linear maps is again linear. So depth adds nothing, and the network cannot even solve XOR. The non-linearity is what makes depth useful.
Derivation.

$$
\begin{aligned}
\mathbf{a}^{[2]} &= W^{[2]}\big(W^{[1]}\mathbf{x} + \mathbf{b}^{[1]}\big) + \mathbf{b}^{[2]} && \text{(two layers, no activation)} \\
&= \big(W^{[2]}W^{[1]}\big)\mathbf{x} + \big(W^{[2]}\mathbf{b}^{[1]} + \mathbf{b}^{[2]}\big) && \text{(multiply out)} \\
&= W\mathbf{x} + \mathbf{b} && \text{(one linear layer)}
\end{aligned}
$$

Summary: No activation → one linear model. No XOR.
Source: DLNN script WS24, Sec. 4.5.

Q2.8 Universal function approximation theorem.

Core. *One hidden layer with enough units can approximate any continuous function on a compact set.*
Explain. The activation must be non-polynomial, for example a sigmoid. The theorem says the network can represent the function. It does not say how many units we need, and it does not say that gradient descent will find the weights.

Summary: One hidden layer, any continuous function. Existence, not learnability.
Source: DLNN I+II script SS25, "Deep Learning theory".

2.3 Backpropagation and the delta error

This is the most asked topic. Practise writing it until it takes two minutes.

Q2.9 Explain backpropagation and define the delta error.

Core. *Backprop gets all gradients with one forward and one backward pass, and the delta is the derivative of the loss with respect to a pre-activation.*
Explain. First the forward pass stores all s and a. Then the deltas are computed from the output back to the input. The gradient of a weight is the delta of the unit above times the activation of the unit below. The cost is linear in the number of weights.
Derivation.

$$\begin{aligned}
\delta_i &:= \frac{\partial L}{\partial s_i} && \text{(definition)}\\
\frac{\partial L}{\partial w_{ij}} &= \frac{\partial L}{\partial s_i}\,\frac{\partial s_i}{\partial w_{ij}} && \text{(chain rule: } w_{ij} \text{ acts only through } s_i)\\
&= \delta_i\, a_j && (s_i = \sum_j w_{ij} a_j + b_i)\\
w_{ij} &\leftarrow w_{ij} - \eta\,\delta_i\, a_j && \text{(gradient step)}
\end{aligned}$$

Summary: δ = dLoss/d(pre-activation). Gradient = δ above $\times$ activation below.

Source: DLNN script WS24, Sec. 4.6.2, Eqs. (4.63)–(4.68).

Q2.10 Write the recursive formula for the deltas.

Core. *The delta of a hidden unit is the weighted sum of the deltas above, times the slope of its own activation.*

Explain. A hidden unit j affects the loss only through the units i of the next layer. So we sum over those paths. In matrix form we multiply by the weight matrix and then by the derivatives of the activations.

Derivation.

$$\begin{aligned}
\delta_j = \frac{\partial L}{\partial s_j} &= \sum_i \frac{\partial L}{\partial s_i}\,\frac{\partial s_i}{\partial a_j}\,\frac{\partial a_j}{\partial s_j} && \text{(chain rule over all units } i \text{ above)}\\
&= \sum_i \delta_i\, w_{ij}\, f'(s_j) && (\partial s_i/\partial a_j = w_{ij},\ \partial a_j/\partial s_j = f'(s_j))\\
\boldsymbol{\delta}^{[l-1]} &= \operatorname{diag}\!\left(f'(\mathbf{s}^{[l-1]})\right) W^{[l]\mathsf{T}}\,\boldsymbol{\delta}^{[l]} && \text{(layer form)}\\
\frac{\partial L}{\partial W^{[l]}} &= \boldsymbol{\delta}^{[l]}\,\mathbf{a}^{[l-1]\mathsf{T}} && \text{(outer product)}
\end{aligned}$$

Summary: Deltas flow down: multiply by W^{T}, then by f'.

Source: DLNN script WS24, Eqs. (4.71), (4.77)–(4.84).

Q2.11 What is the delta at the output with softmax and cross-entropy?

Core. *It is simply prediction minus label.*

Explain. The same clean result holds for a linear output with squared loss and for a sigmoid with binary cross-entropy. That is why these pairs are used together.

Derivation.

$$\begin{aligned}
a_k &= \frac{e^{s_k}}{\sum_m e^{s_m}}, \quad L = -\sum_k y_k \log a_k && \text{(softmax and cross-entropy)}\\
\frac{\partial a_k}{\partial s_i} &= a_k\,(\mathbb{1}[k=i] - a_i) && \text{(derivative of the softmax)}\\
\frac{\partial L}{\partial s_i} &= -\sum_k \frac{y_k}{a_k}\, a_k\,(\mathbb{1}[k=i] - a_i) && \text{(chain rule)}\\
&= -y_i + a_i \sum_k y_k = a_i - y_i && \text{(one-hot label: } \sum_k y_k = 1)
\end{aligned}$$

Summary: Matching pair $\rightarrow$ δ = prediction $-$ label.

Source: DLNN script WS24, Eq. (4.70), Table 4.1.

Q2.12 Vanishing and exploding gradients. Who found it?

Core. *The delta is multiplied by one Jacobian per layer, so over many layers it shrinks or grows exponentially.*
Explain. With the sigmoid the slope is at most 0.25, so the deltas vanish. Sepp Hochreiter analysed this in his diploma thesis in 1991, here in Linz. The fixes are ReLU or SELU, good initialization, normalization, residual connections and, for sequences, the LSTM.
Derivation.

$$\boldsymbol{\delta}^{[1]} = \Big(\prod_{l=2}^{L} \operatorname{diag}(f'(\mathbf{s}^{[l-1]}))\, W^{[l]\mathsf{T}}\Big)\boldsymbol{\delta}^{[L]} \qquad \text{(apply the recursion } L-1 \text{ times)}$$

$$\|\boldsymbol{\delta}^{[1]}\| \le \big(\max|f'| \cdot \max_{l}\|W^{[l]}\|\big)^{L-1}\|\boldsymbol{\delta}^{[L]}\| \qquad \text{(norm of a product)}$$

$$\le \big(0.25\,\|W\|\big)^{L-1}\|\boldsymbol{\delta}^{[L]}\| \qquad \text{(sigmoid: } f' \le 0.25\text{)}$$

If $0.25\|W\| < 1$, the delta vanishes exponentially with depth.

Summary: Product of Jacobians. Hochreiter 1991. Fix: ReLU/SELU, init, ResNet, LSTM.
Source: DLNN script WS24, after Eq. (4.84).

2.4 Gradient descent and optimizers

Q2.13 Write gradient descent. Why mini-batches?

Core. *We move the weights a small step against the gradient of the empirical risk.*
Explain. The full batch gives the exact gradient but is expensive. One sample is cheap but noisy. A mini-batch is the compromise: a good estimate, fast on a GPU, and its noise helps to leave sharp minima.
Derivation.

$$\mathbf{w} \leftarrow \mathbf{w} - \eta\,\nabla_{\mathbf{w}} R_{\text{emp}}(\mathbf{w}) \qquad \text{(full-batch gradient descent)}$$

$$\nabla R_{\text{emp}} \approx \frac{1}{|B|}\sum_{n\in B}\nabla L_n \qquad \text{(mini-batch } B\text{: unbiased estimate)}$$

Summary: Step against the gradient. Mini-batch = cheap, parallel, useful noise.
Source: DLNN script WS24, Sec. 4.10 and Ch. 7.

Q2.14 Which optimizers do you know?

Core. *Gradient descent, SGD, momentum, AdaGrad, RMSprop and Adam.*
Explain. Momentum keeps a running sum of past gradients. AdaGrad and RMSprop give each weight its own step size from the past squared gradients. Adam combines momentum and RMSprop with a bias correction, and it is the usual default.
Derivation.

$$\mathbf{v} \leftarrow \mu\,\mathbf{v} - \eta\,\nabla R_{\text{emp}}, \quad \mathbf{w} \leftarrow \mathbf{w} + \mathbf{v} \qquad \text{(momentum)}$$

$$\mathbf{m} \leftarrow \beta_1\mathbf{m} + (1-\beta_1)\nabla, \quad \mathbf{u} \leftarrow \beta_2\mathbf{u} + (1-\beta_2)\nabla^2 \qquad \text{(Adam: first and second moment)}$$

$$\mathbf{w} \leftarrow \mathbf{w} - \eta\,\frac{\hat{\mathbf{m}}}{\sqrt{\hat{\mathbf{u}}} + \epsilon} \qquad \text{(bias-corrected step)}$$

Summary: GD, SGD, momentum, AdaGrad, RMSprop, Adam.

Source: DLNN script WS24, Ch. 7.

Q2.15 Are there training methods other than gradient descent?

Core. *Yes: closed forms, second-order methods and gradient-free methods.*
Explain. Linear models have a closed form. Logistic regression can use iteratively reweighted least squares. RProp and Quickprop use only the sign or the curvature of the gradient. Newton, Gauss–Newton, Levenberg–Marquardt and conjugate gradients use second-order information. Evolutionary algorithms and random search need no gradient at all.

Summary: Closed form, IRLS, RProp, Newton/LM/CG, evolutionary.

Source: DLNN script WS24, Sec. 4.2–4.3, 7.5–7.6.

2.5 Activation functions, regularization, initialization

Q2.16 Which activation functions do you know? Which one is from Linz?

Core. *Sigmoid, tanh, ReLU and its variants, and ELU and SELU, which were developed at JKU.*
Explain. Sigmoid and tanh are smooth but saturate. ReLU is cheap and does not saturate for positive input, but units can die. SELU (Klambauer et al. 2017) makes a network self-normalizing: the mean and variance of the activations are pulled to 0 and 1. For the output: linear for regression, sigmoid for two classes, softmax for many.
Numbers. SELU: $\lambda \approx 1.0507$, $\alpha \approx 1.6733$, fixed point $(0, 1)$.

Summary: Linz: ELU and SELU. SELU $\rightarrow$ self-normalizing.

Source: DLNN script WS24, Ch. 10.4 and 11.

Q2.17 Which regularization techniques do you know?

Core. *Weight decay, dropout, early stopping, data augmentation and noise.*
Explain. Weight decay adds a penalty on large weights, which shrinks them in every step. Dropout switches units off at random during training, so the network cannot rely on single units. Early stopping ends training when the validation error starts to rise.
Derivation.

$$\tilde{R}(\mathbf{w}) = R_{\text{emp}}(\mathbf{w}) + \tfrac{\lambda}{2}\|\mathbf{w}\|^2 \qquad \text{(weight decay)}$$

$$\nabla \tilde{R} = \nabla R_{\text{emp}} + \lambda\, \mathbf{w} \qquad \text{(gradient of the penalty)}$$

$$\mathbf{w} \leftarrow (1 - \eta\lambda)\, \mathbf{w} - \eta\, \nabla R_{\text{emp}} \qquad \text{(the weights shrink each step)}$$

Summary: Weight decay, dropout, early stopping, augmentation, noise.

Source: DLNN script WS24, Ch. 8.

Q2.18 Why does initialization matter? What is batch normalization?

Core. *Random weights break the symmetry, and their scale must keep the variance constant through the layers.*
Explain. If all weights are equal, all units learn the same thing. If the scale is wrong, the signal and the deltas vanish or explode. Batch normalization standardizes each pre-activation over the mini-batch and then rescales it with learned parameters.
Derivation.

$$s = \sum_{j=1}^{n_{\text{in}}} w_j\, a_j \qquad \text{(one pre-activation)}$$

$$\text{Var}(s) = n_{\text{in}}\, \text{Var}(w)\, \mathbb{E}[a^2] \qquad \text{(independent, zero-mean weights)}$$

$$\text{Var}(w) = \frac{1}{n_{\text{in}}} \qquad \text{(keep } \text{Var}(s) = \mathbb{E}[a^2]\text{)}$$

$$\text{Var}(w) = \frac{2}{n_{\text{in}}} \qquad \text{(ReLU halves } \mathbb{E}[a^2] \text{ (He))}$$

Summary: Break symmetry. Keep variance constant. BN = standardize, then learn scale and shift.

Source: DLNN script WS24, Ch. 9–10.

2.6 Convolutional networks

Q2.19 What is a CNN and why does it work for images?

Core. *Small filters with shared weights slide over the image, so a pattern is found wherever it appears.*

Explain. Images have local structure, and the same pattern can be anywhere. Shared weights give few parameters and translation equivariance. Pooling makes the map smaller and adds some invariance. Deeper layers see larger parts of the image.

Numbers. Output size $\lfloor (n + 2p - k)/s \rfloor + 1$ for input n, kernel k, padding p, stride s.

Summary: Local filters, shared weights, pooling.

Source: DLNN script WS24, Ch. 6.

Q2.20 ResNet, and U-Net for segmentation.

Core. *ResNet adds the input of a block to its output, so gradients pass through very deep networks.*

Explain. The Jacobian of a residual block is the identity plus a small part, so it does not shrink the delta. U-Net is an encoder and a decoder with skip connections. It predicts one class per pixel and copies fine detail across the skips.

Derivation.

$$\mathbf{a}^{[l+1]} = \mathbf{a}^{[l]} + F(\mathbf{a}^{[l]}) \qquad \text{(residual block)}$$

$$\frac{\partial \mathbf{a}^{[l+1]}}{\partial \mathbf{a}^{[l]}} = I + \frac{\partial F}{\partial \mathbf{a}^{[l]}} \qquad \text{(Jacobian stays close to } I\text{)}$$

Summary: ResNet: identity + residual. U-Net: encoder, decoder, skips.

Source: DLNN I+II script SS25, ResNet and semantic segmentation.

2.7 Recurrent networks and LSTM

Q2.21 Explain the forward pass of a recurrent network. How is it trained?

Core. *An RNN reads a sequence step by step and keeps a hidden state that carries the past.*

Explain. The same weights are used at every step. It is trained with backpropagation through time: unfold the network over the sequence and backpropagate as in a deep network with shared weights. The number of parameters does not grow with the length of the sequence.

Derivation.

$$\mathbf{s}(t) = W\mathbf{x}(t) + R\,\mathbf{a}(t-1), \quad \mathbf{a}(t) = f(\mathbf{s}(t)) \qquad \text{(forward pass)}$$
$$\boldsymbol{\delta}(t) = \operatorname{diag}(f'(\mathbf{s}(t)))\Big(V^{\mathsf{T}}\frac{\partial L(t)}{\partial \hat{\mathbf{y}}(t)} + R^{\mathsf{T}}\boldsymbol{\delta}(t+1)\Big) \qquad \text{(delta from the output and from the next step)}$$
$$\frac{\partial L}{\partial R} = \sum_t \boldsymbol{\delta}(t)\,\mathbf{a}(t-1)^{\mathsf{T}} \qquad \text{(shared weights: sum over time)}$$

Summary: Same weights every step. Hidden state = memory. BPTT = backprop on the unfolded net.

Source: LSTM lecture WS24 (Hochreiter).

Q2.22 Why do plain RNNs fail on long sequences, and how does LSTM fix it?

Core. *The delta is multiplied by the recurrent Jacobian at every step, so it vanishes; the LSTM cell is updated by addition instead.*

Explain. The LSTM (Hochreiter and Schmidhuber 1997) has a memory cell with a constant error carousel. Gates decide what to write, what to keep and what to read. With the forget gate near one, the error flows back through time without shrinking.

Derivation.

$$\mathbf{c}(t) = \mathbf{f}(t) \odot \mathbf{c}(t-1) + \mathbf{i}(t) \odot \mathbf{z}(t) \qquad \text{(additive cell update)}$$
$$\mathbf{h}(t) = \mathbf{o}(t) \odot \tanh(\mathbf{c}(t)) \qquad \text{(output gate reads the cell)}$$
$$\frac{\partial \mathbf{c}(t)}{\partial \mathbf{c}(t-1)} = \operatorname{diag}(\mathbf{f}(t)) \qquad \text{(direct path through the cell)}$$
$$\approx I \qquad \text{(forget gate near one: error carried unchanged)}$$

Summary: Additive cell = constant error carousel. Gates: input, forget, output.

Source: LSTM lecture WS24, slides 6–10.

Q2.23 Bidirectional RNNs, and one label for a whole sequence.

Core. *A bidirectional RNN runs one RNN forward and one backward and joins their states.*

Explain. So every step sees the past and the future. It needs the whole sequence, so it is not for real-time use. For one label per sequence we use the last hidden state, or we pool all hidden states and put a classifier on top.

Summary: Forward + backward, concatenate. Sequence label: last state or pooling.

Source: LSTM lecture WS24.

2.8 Attention and Transformers

Q2.24 Explain attention and the Transformer.

Core. *Every token takes a weighted mean of all tokens, and the weights come from query times key.*

Explain. Each token makes a query, a key and a value by linear maps. The softmax of query times key gives the weights, and the output mixes the values with them. A Transformer block is multi-head self-attention plus an MLP, each with a residual connection and layer normalization.

Derivation.

$$Q = XW_Q,\ K = XW_K,\ V = XW_V \qquad \text{(linear maps of the tokens)}$$

$$\mathrm{Att}(Q,K,V) = \mathrm{softmax}\Big(\frac{QK^\mathsf{T}}{\sqrt{d_k}}\Big)V \qquad \text{(weights per row sum to one)}$$

$$\mathrm{Var}(\mathbf{q}\cdot\mathbf{k}) = \sum_{m=1}^{d_k} \mathrm{Var}(q_m k_m) = d_k \qquad \text{(independent entries of variance 1)}$$

$$\mathrm{Var}(\mathbf{q}\cdot\mathbf{k}/\sqrt{d_k}) = 1 \qquad \text{(so the softmax does not saturate)}$$

Summary: Query meets key → softmax weights → mix the values.

Source: LSTM lecture WS24, slides 12–15.

Q2.25 Transformers are permutation-equivariant. Why is that a problem?

Core. *Self-attention sees the tokens as a set, so the order of words is lost.*

Explain. If we shuffle the tokens, the outputs are shuffled the same way. So we add the position to each token: fixed sinusoids, learned vectors, or relative encodings such as RoPE.

Summary: Attention sees a set. Add position information.

Source: Past-student report; LSTM lecture WS24.

Q2.26 Compare RNN and Transformer in cost.

Core. *An RNN is linear in the sequence length but sequential; a Transformer is quadratic but parallel.*

Explain. In a Transformer every token attends to every token, so time and memory grow with N^2. But all tokens are processed at once, and long-range paths are one step long. LSTM-type models such as xLSTM are attractive again because inference is linear in N.

Numbers. RNN $O(N)$, Transformer $O(N^2)$.

Summary: RNN: linear, sequential. Transformer: quadratic, parallel.

Source: Past-student report; LSTM lecture WS24.

2.9 Classical machine learning in the same slot

Q2.27 Explain the support vector machine.

Core. *The SVM finds the separating hyperplane with the largest margin.*

Explain. The margin is $2/\|\mathbf{w}\|$, so maximizing it means minimizing $\|\mathbf{w}\|^2$. The soft margin allows some points inside the margin through slack variables and a penalty C. In the dual only dot products appear, and the points with $\alpha_n > 0$ are the support vectors.

Derivation.

$$\min_{\mathbf{w},b,\boldsymbol{\xi}} \ \tfrac{1}{2}\|\mathbf{w}\|^2 + C\sum_n \xi_n \qquad \text{(large margin, few violations)}$$

$$\text{s.t.}\ \ y_n(\mathbf{w}^\mathsf{T}\mathbf{x}_n + b) \geq 1 - \xi_n,\quad \xi_n \geq 0 \qquad \text{(every point on the right side, up to slack)}$$

$$\max_{\boldsymbol{\alpha}} \ \sum_n \alpha_n - \tfrac{1}{2}\sum_{n,m} \alpha_n\alpha_m y_n y_m\, \mathbf{x}_n^\mathsf{T}\mathbf{x}_m \qquad \text{(dual, } 0 \leq \alpha_n \leq C,\ \textstyle\sum_n \alpha_n y_n = 0\text{)}$$

Summary: Max margin. Dual has only dot products.

Source: ML: Supervised Techniques, SVM unit.

Q2.28 What is a kernel and the kernel trick?

Core. *A kernel is a dot product in a feature space, computed without building the features.*
Explain. The SVM dual needs only dot products. We replace them by a kernel and get a non-linear classifier at the cost of a linear one. Examples are the polynomial kernel and the Gaussian kernel.
Derivation.

$$\begin{aligned} k(\mathbf{x},\mathbf{x}') &= (\mathbf{x}^\mathsf{T}\mathbf{x}')^2 && \text{(polynomial kernel in 2D)} \\ &= x_1^2x_1'^2 + 2x_1x_2x_1'x_2' + x_2^2x_2'^2 && \text{(multiply out)} \\ &= \phi(\mathbf{x})^\mathsf{T}\phi(\mathbf{x}'), \quad \phi(\mathbf{x}) = (x_1^2, \sqrt{2}\,x_1x_2, x_2^2) && \text{(a dot product of features)} \end{aligned}$$

Summary: Kernel = dot product in feature space, without computing the features.
Source: ML: Supervised Techniques, kernels unit.

Q2.29 What is the VC dimension?

Core. *It is the largest number of points that a model class can label in every possible way.*
Explain. It measures capacity. Linear classifiers in d dimensions have VC dimension $d+1$. In generalization bounds the gap between training and test error grows with the VC dimension and shrinks with the number of samples.
Numbers. Linear classifier in $\mathbb{R}^d$: $d+1$.
Summary: Largest set it can shatter. Linear: $d+1$.
Source: ML: Supervised Techniques; statistical learning theory.

Q2.30 Likelihood, maximum likelihood, and its problems.

Core. *Maximum likelihood chooses the parameters that make the observed data most probable.*
Explain. We maximize the log-likelihood, because a sum is easier than a product. With little data it overfits: an event never seen gets probability zero. A prior (MAP) or regularization fixes this. Squared loss is maximum likelihood under Gaussian noise.
Derivation.

$$\begin{aligned} p(y \mid \mathbf{x}) &= \mathcal{N}(y;\, g(\mathbf{x};\mathbf{w}),\, \sigma^2) && \text{(Gaussian noise model)} \\ -\log \prod_n p(y_n \mid \mathbf{x}_n) &= N\log(\sqrt{2\pi}\,\sigma) + \frac{1}{2\sigma^2}\sum_n \big(y_n - g(\mathbf{x}_n;\mathbf{w})\big)^2 && \text{(log of the Gaussian)} \\ \arg\max_{\mathbf{w}}\ \text{likelihood} &= \arg\min_{\mathbf{w}}\ \sum_n \big(y_n - g(\mathbf{x}_n;\mathbf{w})\big)^2 && \text{(first term has no } \mathbf{w}) \end{aligned}$$

Summary: Most likely parameters. Overfits with little data → prior.
Source: DLNN script WS24, Eqs. (4.8)–(4.11).

Q2.31 Derive the REINFORCE loss. (Asked by Prof. Hochreiter.)

Core. *The log-derivative trick moves the gradient inside the expectation, so we can estimate it from sampled episodes.*
Explain. We want the largest expected return. The environment does not depend on the policy parameters, so only the policy terms survive. As a loss we minimize minus log-probability times return, with a baseline to reduce the variance.
Derivation.

$$J(\theta) = \int p_\theta(\tau)\, G(\tau)\, \mathrm{d}\tau \qquad \text{(expected return over trajectories)}$$

$$\nabla_\theta J = \int p_\theta(\tau)\, \nabla_\theta \log p_\theta(\tau)\, G(\tau)\, \mathrm{d}\tau \qquad (\nabla p = p\, \nabla \log p)$$

$$\log p_\theta(\tau) = \log p(s_0) + \sum_t \left[\log \pi_\theta(a_t \mid s_t) + \log p(s_{t+1} \mid s_t, a_t)\right] \qquad \text{(trajectory probability)}$$

$$\nabla_\theta J = \mathbb{E}_\tau\Big[\sum_t \nabla_\theta \log \pi_\theta(a_t \mid s_t)\, G_t\Big] \qquad \text{(dynamics do not depend on } \theta)$$

$$L(\theta) = -\sum_t \log \pi_\theta(a_t \mid s_t)\, (G_t - b) \qquad \text{(loss with baseline } b)$$

Summary: Log-derivative trick. Loss $= -\log \pi(a \mid s) \times$ return.

Source: Deep Reinforcement Learning VL, policy gradient unit.

Q2.32 How do you evaluate a classifier?

Core. *With the confusion matrix, precision, recall and the area under the ROC curve.*

Explain. Precision: of what I called positive, how much is right. Recall: of all positives, how many I found. The ROC curve plots the true-positive rate against the false-positive rate over all thresholds. The AUC is the probability that a random positive gets a higher score than a random negative.

Numbers. Precision $TP/(TP + FP)$. Recall $TP/(TP + FN)$. AUC 0.5 = chance, 1 = perfect.

Summary: Precision: how clean. Recall: how complete. AUC: ranking quality.

Source: ML: Supervised Techniques.

Q2.33 How else can you measure robustness, besides adversarial attacks?

Core. *Test under distribution shift and check how sure the model is.*

Explain. Use corrupted inputs, new domains or data from another time. Check calibration and uncertainty with ensembles or MC dropout, and the sensitivity of the output to the input. My thesis has a direct example: a recording made under one boundary condition fails under another, because the regime changed.

Summary: Corruptions, domain shift, calibration, sensitivity, out of distribution.

Source: DLNN script WS24, Sec. 4.8; thesis, Section 6.4.

3 AI for Simulation (Prof. Pirker)

Prof. Pirker is the supervisor and taught *Numerical Methods in Fluid Mechanics*. Expect him to stay close to recurrence CFD and to the figures of the thesis: he points at a figure and asks what it shows and why. This part has three blocks: rCFD as taught in the lecture, the figures of the thesis, and the CFD basics that rCFD stands on.

3.1 Recurrence CFD as taught in the lecture

Q3.1 Why rCFD at all? Connect it to how Navier–Stokes is solved.

Core. *rCFD makes a bold prediction, reusing recorded flow, and then corrects it with physics, just as SIMPLE guesses a pressure and corrects it.*
Explain. Many industrial flows run for a long time and show recurring patterns: bubble columns, fluidized beds, turbulent vessels. Solving momentum and pressure for all that time is wasteful once the flow repeats itself. The correction then fixes what the reuse breaks, for example the mass balance.

Summary: Bold prediction + physics-based correction.

Source: Pirker, rCFD lecture, slides 16–18.

Q3.2 Flow-based versus transport-based rCFD.

Core. *Flow-based rCFD stitches recorded flows and still solves a transport equation; transport-based rCFD replaces the equation by cell shifts and face swaps.*
Explain. Flow-based rCFD (Lichtenegger and Pirker 2016) builds a long artificial flow from recorded sequences, about 100 times faster. Transport-based rCFD (Pirker and Lichtenegger 2018) solves no transport equation at all. My thesis uses the transport-based form, plus the correction hook and the conservation constraint.
Numbers. Steel ladle: 26 h of full CFD against 1.6 s of transport-based rCFD.

Summary: Flow-based: stitch flows, solve transport. Transport-based: shifts and swaps.

Source: Pirker, rCFD lecture, slides 19–34.

Q3.3 What are the recurrence norm, matrix, plateau and prominence?

Core. *The norm measures how different two states are, and the prominence measures how deep the flow dips back towards earlier states.*
Explain. The norm for all pairs gives the matrix. One row of it starts at zero on the diagonal, rises and then oscillates about a plateau. Deep dips below the plateau mean the flow comes back to similar states, so the replay can jump there. In my tank the dips are shallow, so the replay plays the frames in order.
Numbers. The method asks for a prominence of 0.05 to 0.1. My tank: median 0.028.
Derivation.

$$
\begin{aligned}
R_{fg} &= \left\| \Phi^{(f)} - \Phi^{(g)} \right\| && \text{(norm between two frames)} \\
d(f,g) &= R_{fg} / \max R && \text{(normalised distance)} \\
P_{\mathrm{rel}} &= \langle d_{\max} - d_{\min} \rangle / \bar{d} && \text{(dip depth over the plateau)}
\end{aligned}
$$

Summary: Norm, matrix, row, plateau, prominence. Mine: 0.028, frames in order.

Source: Pirker, rCFD lecture, slides 50–51; thesis, Section 2.4.

Q3.4 The three "bold failures" of rCFD and their corrections.

Core. *Mass imbalance, unphysical diffusion and too little recurrence, fixed by a global balance, local swaps and recurrence islands.*
Explain. First, cell shifts do not conserve mass locally, so a global monitor corrects the total with mass-acting face swaps. Second, a fixed number of swaps gives the same mixing everywhere, so the swaps should follow the local mixing. Third, in large domains frames that match globally do not match locally, so the domain is split into islands with their own matrices. My thesis touches all three: the conservation constraint, the recorded diffusivity, and a flow with too little prominence.

Summary: Imbalance → global balance. Diffusion → local swaps. Low recurrence → islands.

Source: Pirker, rCFD lecture, slides 35–54; thesis, Sections 2.6–2.8.

Q3.5 When does rCFD work, and when does it fail?

Core. *It works when the motion repeats and the scalar does not change it, and it fails when the scalar drives the flow or the operating state changes.*
Explain. In a new state the recording does not contain the motion that this state would produce. That is my result: under the cooled lid the flow keeps renewing itself with cold plumes, a motion that the adiabatic recording never saw. The answer of the lecture is multi-state rCFD: one database per process state.

Summary: A database belongs to a flow regime, not to a case.

Source: Pirker, rCFD lecture, slide 59; thesis, Section 6.4, Figure 6.6.

Q3.6 rCFD and digital twins. How would you use your work in a plant?

Core. *rCFD can run alongside the process, if live data keep aligning it.*
Explain. A tank model with one mean value is instant but ignores the inside of the plant. Full CFD shows the inside but takes days. rCFD sits in between. Run blindly forward, it collects error, so live data should correct it. My thesis shows what such a twin must watch: a boundary condition can move the process into a regime the database does not cover.
Numbers. Lecture example: tank model under 1 s, experiment 750 s, CFD 2 days, rCFD 30 s.

Summary: Fast but blind, exact but slow, rCFD in between. Align with live data.

Source: Pirker, rCFD lecture, slides 59–72.

Q3.7 Is rCFD machine learning?

Core. *It is data-based simulation: an expensive ground truth makes data, a cheap model reuses it, and the question is how far it generalizes.*
Explain. There is no gradient training. Choosing the next frame is a nearest-neighbour search. The replay is a sequence of sparse linear operators, close to a learned propagator (Lumetzberger et al. 2025). Possible AI steps: learn the map for unseen states, learn the missing flux as a function of the state, or train a neural operator on the database.

Summary: Ground truth → data → cheap reuse. Generalization = which regime the database covers.

Source: Pirker, rCFD lecture, slide 69; thesis, Section 2.2.

3.2 The figures of the thesis

For every figure, have three sentences ready: what is plotted, what we see, and why.

Q3.8 Figure 1.1 – passive and non-passive transport.

Core. *A dye only rides the flow, but temperature makes the flow, and the replay cuts this feedback.*
Explain. (a) A dye in the stored velocity of the adiabatic tank is only stretched. (b) In the cooled tank, cold plumes from the lid create the motion. (c) The replay moves temperature with recorded maps and never solves the velocity.

Summary: The whole thesis question in one figure.

Q3.9 Figure 2.2 – recurrence of the database.

Core. *The flow has two phases, filling and settling, and no state comes back.*
Explain. The distance matrix has two blocks split at frame 315. The rows rise to a plateau and none returns towards zero. The prominence stays below what the method needs, so the replay plays the frames in order.
Numbers. Median prominence 0.028. Nearest-neighbour distance 0.013, only because neighbouring frames are similar.

Summary: Two blocks, no return, frames in order.

Q3.10 Figure 3.3 – vertical heat transport and plume scale.

Core. *Under the cooled lid, the flow carries heat about 600 times more than conduction.*
Explain. The plane-mean vertical heat flux is split into the part carried by the flow and the part carried by conduction. The temperature departures under the lid have the size of whole plumes. So cooling is a transport process, and a diffusive step cannot copy it.

Summary: Cooling is transport, not diffusion.

Q3.11 Figures 3.4 and 3.5 – the two regimes.

Core. *One boundary condition switches the flow from one regime to another.*
Explain. With the adiabatic lid the jet rises, spreads under the lid, and the warm layer then rests. With the cooled lid the layer above the front overturns in a row of cells driven by cold plumes. Same geometry, same inflow, two different flows.

Summary: Same tank, same inflow, two flows.

Q3.12 Figure 5.1 – the standard replay over the face-exchange sweep.

Core. *More mixing heats the tank faster during the inflow and drains it faster afterwards.*
Explain. The mean temperature and the height of the heat are shown for seven strengths. The height is best at intermediate strengths, the mean temperature with little mixing. So no single strength is best for both.

Summary: No single best f_{sd}.

Q3.13 Figure 5.2 – the adiabatic tank through the run.

Core. *The recorded maps keep the front thin, the face exchange smears it, and the correction makes it thin again.*
Explain. The mid-plane temperature is shown at four times, with the front width in each panel. The replay with maps only keeps the front as thin as the reference. So the smearing comes from the exchange, not from the maps.

Summary: Maps: thin. Exchange: smeared. Correction: thin again.

Q3.14 Figure 5.3 – space–time diagram.

Core. *It shows the warm layer growing downwards and where the standard replay is too warm or too cold.*

Explain. The horizontal mean temperature is plotted against height and time. The difference panel shows the error of the standard replay, and the thickness panel shows that its front stays too thick. The corrected replay follows the reference.

Summary: Height against time: the layer grows, the error is visible.

Q3.15 Figure 5.6 – every correction run in one map.

Core. *Only the vertically weighted form brings both errors down while the front reaches the reference width.*

Explain. Each marker is one replay: trace error and end-profile error against the front width. The other forms reach a thin front only with a large error. The working range of the vertical form is $s = 0.40$ to 0.50.

Numbers. Reference front 5.0 mm.

Summary: Direction matters more than strength.

Q3.16 Figure 5.7 – buoyancy frequency across the front.

Core. *A sharper front is a stronger barrier to vertical motion.*

Explain. The buoyancy frequency measures how strongly the stratification resists vertical motion. The standard replay holds a barrier about half as strong as the reference. The corrected replay reaches the reference value after the inflow.

Derivation.

$$N^2 = g\,\beta\,\frac{\partial T}{\partial y} \qquad \text{(buoyancy frequency, Eq. (3.3))}$$

Summary: Steeper gradient, larger N, stronger barrier.

Q3.17 Figure 5.8 – end profiles through the front.

Core. *Read the width always together with the profile.*

Explain. Without correction the front is wide and too high. The unweighted correction makes it thin but puts it in the wrong place. The vertical weight follows the reference. A saw-toothed front can have the right width and the wrong shape.

Summary: Right width is not enough: check the shape.

Q3.18 Figure 5.10 – outlet temperature from the energy balance.

Core. *The outlet sits just below the front, so a smeared front turns into a heat leak.*

Explain. While the inflow is warm the standard replay keeps heat in the tank, and afterwards it drains it. The vertical correction ends much closer. So a numerical error at the front becomes an error in the energy budget.

Numbers. Standard: $+2.7$ K kept at $t = 315$ s, ends at -1.9 K. Vertical correction ends at -0.5 K.

Derivation.

$$M\frac{\mathrm{d}\bar{T}}{\mathrm{d}t} = \dot{m}\,(T_{\mathrm{in}} - T_{\mathrm{out}}) \qquad \text{(energy in minus out, Eq. (4.10))}$$

$$T_{\mathrm{out}} = T_{\mathrm{in}} - \tau\,\frac{\mathrm{d}\bar{T}}{\mathrm{d}t}, \quad \tau = M/\dot{m} \qquad \text{(solve for the outlet, Eq. (4.11))}$$

Summary: Smeared front + outlet below it = heat leak.

Q3.19 Figure 5.11 – state space.

Core. *The correction keeps the replay close to the states that the recording spans.*
Explain. The two leading POD modes of the reference field are shown, and the distance of each run from the space of the first ten modes. The standard replay leaves this space and does not return. The corrected replay returns after the inflow.

Summary: Corrected replay stays on the recorded state space.

Q3.20 Figure 6.2 – wall-cooled end profiles.

Core. *The standard replay holds nearly the right heat, but in the wrong place.*
Explain. The bulk is too warm and the top layer too cold. The correction moves the heat back. This is why I report the mean temperature and the profile together.
Numbers. Standard replay: profile 2.79 K. Corrected at $s = 0.475$: 0.59 K.

Summary: Same heat, wrong place.

Q3.21 Figure 6.3 – wall-cooled traces.

Core. *The local correction follows the heat content but lags in layering while the tank fills.*
Explain. The red band shows how far the local correction lags in heterogeneity during the warm inflow. Only the recorded flux also follows the heterogeneity. A lag in layering is missing transport, not missing sharpness.

Summary: Layering lag = missing transport.

Q3.22 Figures 6.4 and 6.5 – end state and plumes.

Core. *The departure is smallest at $s = 0.55$, and only the corrected replay has plumes that reach the front.*
Explain. Along the strength ladder the cold top and the warm bulk fade together. At $s = 0.8$ the top turns warm. In 3D, the plumes of the standard replay end halfway down, and with the correction they reach the front again. The plumes were in the recorded maps all along.
Numbers. Smallest mean departure 0.22 K at $s = 0.55$.

Summary: Right layering lets the recorded plumes show.

Q3.23 Figure 6.6 – recurrence of the two recordings.

Core. *After the inflow the cooled flow keeps renewing itself, and the adiabatic flow hardly changes.*
Explain. At short lags the cooled flow changes about ten times faster. So the cooled case is a different regime, and a database recorded without cooling cannot contain its motion.

Summary: Different regime → different database.

3.3 CFD basics behind rCFD

Q3.24 Reynolds number and other dimensionless numbers.

Core. *The Reynolds number compares inertia with viscous forces.*
Explain. Pipe flow is laminar below about 2300 and turbulent above about 4000. Other numbers: the Péclet number compares convection with diffusion, the Prandtl number momentum diffusion with heat diffusion, the Froude number inertia with gravity, and the Richardson number buoyancy with inertia.
Numbers. My case: $Re \approx 99$, laminar. $Ri = 1.84$.
Derivation.

$$Re = \frac{UL}{\nu}, \quad Pe = \frac{UL}{\Gamma}, \quad Pr = \frac{\nu}{\alpha}, \quad Ri = \frac{g'L}{U^2} \qquad \text{(ratios of effects)}$$

Summary: Re: inertia/viscosity. Pe: convection/diffusion. Ri: buoyancy/inertia.

Source: Pirker, introduction slides; thesis, Section 3.3.

Q3.25 Governing equations and the material derivative.

Core. *Mass, momentum and energy are conserved, and the material derivative links the moving parcel with the fixed control volume.*

Explain. The Lagrangian view follows a fluid parcel. The Eulerian view looks at a fixed point. The material derivative is the rate of change seen by the moving parcel.

Derivation.

$$\begin{aligned} \frac{\mathrm{d}}{\mathrm{d}t}\phi(\mathbf{x}(t),t) &= \frac{\partial\phi}{\partial t} + \sum_i \frac{\partial\phi}{\partial x_i}\frac{\mathrm{d}x_i}{\mathrm{d}t} && \text{(chain rule along the path)} \\ &= \frac{\partial\phi}{\partial t} + \mathbf{u}\cdot\nabla\phi = \frac{\mathrm{D}\phi}{\mathrm{D}t} && \text{(the parcel moves with } \mathrm{d}\mathbf{x}/\mathrm{d}t = \mathbf{u}) \\ \frac{\partial\rho}{\partial t} + \nabla\cdot(\rho\mathbf{u}) &= 0 && \text{(mass)} \\ \frac{\partial(\rho\mathbf{u})}{\partial t} + \nabla\cdot(\rho\mathbf{u}\mathbf{u}) &= -\nabla p + \nabla\cdot\boldsymbol{\tau} + \rho\mathbf{g} && \text{(momentum)} \end{aligned}$$

Summary: Change in time + convection = pressure + viscous + body force.

Source: Pirker, governing-equation slides.

Q3.26 The finite-volume method.

Core. *Integrate the equation over each cell, and the divergence theorem turns everything into fluxes through faces.*

Explain. What leaves one cell enters its neighbour, so the method is conservative. Each cell value is linked to its neighbours by one linear equation. If all coefficients are positive, the new value is a weighted mean, and no new extremum can appear. The same rule protects the face exchange in rCFD.

Derivation.

$$\begin{aligned} \frac{\partial c}{\partial t} + \nabla\cdot(\mathbf{u}c) &= \nabla\cdot(\Gamma\nabla c) && \text{(transport equation)} \\ V\frac{\mathrm{d}c_P}{\mathrm{d}t} + \oint \mathbf{u}c\cdot\mathbf{n}\,\mathrm{d}A &= \oint \Gamma\,\nabla c\cdot\mathbf{n}\,\mathrm{d}A && \text{(integrate, divergence theorem)} \\ a_P\,c_P &= \sum_{NB} a_{NB}\,c_{NB} + b && \text{(fluxes per face: the computational molecule)} \end{aligned}$$

Summary: Integrate, fluxes through faces, conservative, positive coefficients.

Source: Pirker, finite-volume slides.

Q3.27 Discretization schemes and numerical diffusion.

Core. *First-order upwind is stable but adds false diffusion; central differencing is more accurate but can oscillate.*

Explain. Upwind takes the value from upstream. Its truncation error acts like an extra diffusivity that smears sharp gradients. Central differencing oscillates when the cell Péclet number is above

2. Finer or flow-aligned grids and higher-order schemes reduce numerical diffusion. My thesis is a numerical-diffusion story: the face exchange smears the front, and the correction undoes it.
Derivation.

$$
\begin{aligned}
c_{i-1} &= c_i - \Delta x\,\partial_x c + \tfrac{1}{2}\Delta x^2\,\partial_{xx} c - \dots && \text{(Taylor series)}\\
\frac{c_i - c_{i-1}}{\Delta x} &= \partial_x c - \tfrac{1}{2}\Delta x\,\partial_{xx} c + \dots && \text{(upwind difference, } u>0\text{)}\\
\partial_t c + u\,\partial_x c &= \tfrac{1}{2} u\Delta x\,\partial_{xx} c + \dots && \text{(what upwind really solves)}\\
\Gamma_{\text{num}} &= \tfrac{1}{2}\,u\,\Delta x && \text{(numerical diffusivity)}
\end{aligned}
$$

Summary: Upwind: stable, diffusive, $\Gamma_{\text{num}} = u\Delta x/2$. Central: accurate, oscillates if $Pe > 2$.
Source: Pirker, finite-volume slides II.

Q3.28 Explicit versus implicit time stepping; Courant and diffusion numbers.

Core. *Explicit steps are cheap but limited by stability; implicit steps solve a system but are unconditionally stable.*
Explain. Explicit convection needs a Courant number below one: information must not travel more than one cell per step. Explicit diffusion needs a diffusion number below one half. This is exactly the cap that stopped the recorded diffusivity in my replay.
Derivation.

$$
\begin{aligned}
c_i^{n+1} &= c_i^n + d\,(c_{i+1}^n - 2c_i^n + c_{i-1}^n), \quad d = \frac{\Gamma\,\Delta t}{\Delta x^2} && \text{(explicit diffusion)}\\
G &= 1 - 4d\,\sin^2(k\Delta x/2) && \text{(insert a Fourier mode)}\\
|G| \le 1 &\iff 4d \le 2 \iff d \le \tfrac{1}{2} && \text{(stable for every mode)}
\end{aligned}
$$

Numbers. $Co = u\Delta t/\Delta x < 1$. $d < 0.5$.
Summary: $Co < 1$, $d < 0.5$. My cap is the explicit diffusion limit.
Source: Pirker, finite-volume slides; thesis, Section 5.2.1.

Q3.29 The SIMPLE algorithm.

Core. *Guess the pressure, solve the velocity, correct the pressure from continuity, and repeat.*
Explain. SIMPLE stands for Semi-Implicit Method for Pressure-Linked Equations (Patankar). The velocity correction depends on the pressure correction. Putting it into continuity gives an equation for the pressure correction. Then both fields are corrected, with under-relaxation, until the residuals are small.
Derivation.

$$
\begin{aligned}
&\mathbf{u}^* \leftarrow \text{momentum with } p^* && \text{(1: predict the velocity)}\\
&u' = d\,(p'_P - p'_E) && \text{(2: velocity correction from pressure correction)}\\
&\nabla\cdot(\mathbf{u}^* + \mathbf{u}') = 0 \Rightarrow p' && \text{(3: continuity gives } p'\text{)}\\
&p = p^* + \alpha_p\,p', \quad \mathbf{u} = \mathbf{u}^* + \mathbf{u}' && \text{(4: correct, then repeat)}
\end{aligned}
$$

Summary: Guess p, solve u, correct from continuity, repeat.
Source: Pirker, finite-volume slides.

Q3.30 Turbulence: phenomena, Reynolds decomposition, models.

Core. *Turbulence is chaotic and mixes strongly; RANS models all eddies, LES models only the small ones, DNS models none.*
Explain. Energy enters at the large eddies and cascades down to the Kolmogorov scale, where viscosity dissipates it. RANS splits the velocity into a mean and a fluctuation, and the Reynolds stresses need a model, usually an eddy viscosity. My case needs none of this, because it is laminar.
Derivation.

$$u = \bar{u} + u' \qquad \text{(Reynolds decomposition)}$$
$$\nu_t = C_\mu \, k^2/\varepsilon \qquad (k\text{–}\varepsilon \text{ eddy viscosity})$$
$$\eta = \nu^a \varepsilon^b : \quad \mathrm{m} = (\mathrm{m}^2/\mathrm{s})^a (\mathrm{m}^2/\mathrm{s}^3)^b \qquad \text{(Kolmogorov scale from } \nu \text{ and } \varepsilon \text{ only)}$$
$$\Rightarrow \; 2a + 2b = 1, \quad a + 3b = 0 \qquad \text{(match metres and seconds)}$$
$$\eta = (\nu^3/\varepsilon)^{1/4} \qquad (a = 3/4,\ b = -1/4)$$

Summary: Cascade to η. RANS all, LES small, DNS none.
Source: Pirker, turbulence slides.

Q3.31 Wall boundary layers, y^+ and wall functions.

Core. *Near a wall the velocity drops to zero, and y^+ says how far the first cell sits from the wall in viscous units.*
Explain. The viscous sublayer lies below $y^+ \approx 5$ and the log layer above about 30. Wall functions place the first cell in the log layer and model the rest, which is cheap. A wall-resolved simulation needs $y^+ \approx 1$ at the first cell.
Derivation.

$$u_\tau = \sqrt{\tau_w/\rho}, \quad y^+ = \frac{u_\tau \, y}{\nu} \qquad \text{(friction velocity, wall distance in viscous units)}$$

Summary: $y^+ < 5$ viscous, > 30 log layer. Wall functions: first cell in the log layer.
Source: Pirker, turbulence slides III.

Q3.32 Species and heat transport; the Boussinesq approximation.

Core. *Temperature and species follow the same transport equation, and Boussinesq keeps the density constant except where gravity acts.*
Explain. If the density depends on temperature, flow and temperature are coupled. For small density changes we keep the density constant everywhere except in the gravity term. In my case the change is 1.5 per cent.
Derivation.

$$\frac{\partial c}{\partial t} + \nabla \cdot (\mathbf{u} c) = \nabla \cdot (\Gamma \nabla c) + S \qquad \text{(scalar transport)}$$
$$\rho \, \mathbf{g} \approx \rho_0 \, \mathbf{g}(1 - \beta(T - T_0)) \qquad \text{(Boussinesq: density only in the buoyancy term)}$$

Summary: Constant density, except where gravity sees it.
Source: Pirker, species and heat slides; thesis, Section 2.1.

Q3.33 Multiphase flows: Euler–Euler versus Euler–Lagrange.

Core. *Euler–Euler treats each phase as a continuum; Euler–Lagrange tracks every particle.*
Explain. Euler–Euler uses volume fractions and suits dense systems at lower cost. Euler–Lagrange follows particles in the fluid, and with particle contacts it becomes CFD–DEM: accurate but expensive. rCFD was first used for bubble columns and fluidized beds.

Summary: Continuum phases versus tracked particles.

Source: Pirker, multiphase slides.